

Talk Sarasavi — Architecture Blueprint

Prepared May 2026 | Version 1.1 | Pre-implementation reference document

1. Guiding Principles

Every architecture decision in this document is governed by three rules:

1. **Single source of truth** — Logic lives in one place. Business rules live in Cloud Functions, prompts live in Remote Config, data lives in Firestore.
2. **Language-pair agnostic from day one** — `"Sinhala"` is never a hardcoded string. It is always a config variable (`si → en`). Adding Tamil → English later is a config change, not a code change.
3. **Server enforces, client displays** — The server is the authority on usage limits, subscription status, and AI responses. The client's only job is to capture audio, play audio, and render UI.

2. Monorepo Structure

Simple flat folder structure. No Nx, no Turborepo — just disciplined naming.

```

talk-sarasavi/
|
├─ apps/
|   └─ mobile/                                # React Native Expo app (iOS + Android)
|
├─ backend/
|   ├── functions/                          # Firebase Cloud Functions (TypeScript)
|   ├── firestore/
|   |   ├── firestore.rules                 # Security rules
|   |   └─ firestore.indexes.json          # Composite indexes
|   └─ storage/
|       └─ storage.rules
|
├─ shared/
|   ├── contracts/                          # API contract definitions (plain TypeScript types)
|   |   ├── api.types.ts                   # Request/response shapes shared as docs
|   |   └─ error.codes.ts                  # Canonical error codes used across all platforms
|   |
|   ├── docs/
|   |   ├── architecture/                 # This document and supporting diagrams
|   |   ├── api/                          # API endpoint reference
|   |   └─ data-models/                   # Firestore schema reference
|   |
|   └─ scripts/
|       ├── deploy.sh                     # Full deployment script
|       └─ seed-dev.sh                    # Seed Firestore emulator with test data
|
├─ .github/
|   └─ workflows/
|       ├── deploy-functions.yml           # CI: deploy Cloud Functions on merge to main
|       └─ lint-check.yml                  # CI: lint all platforms on every PR
|
├─ firebase.json                          # Firebase project config (project: talk-sarasavi)
├─ .firebaserc                             # Firebase project aliases (dev / prod)
├─ .gitignore
└─ README.md

```

3. Firestore Data Schema

3.1 Collection: `users/{userId}`

Top-level user document. Created on first sign-in.

Field	Type	Description
<code>uid</code>	<code>string</code>	Firebase Auth UID
<code>displayName</code>	<code>string</code>	User's chosen name
<code>createdAt</code>	<code>timestamp</code>	Account creation time
<code>lastActiveAt</code>	<code>timestamp</code>	Last session end time
<code>languagePair</code>	<code>object</code>	<code>{ nativeLanguage: "si", targetLanguage: "en" }</code>
<code>currentLevel</code>	<code>number</code>	0–100 score representing English proficiency
<code>streakDays</code>	<code>number</code>	Current consecutive day streak
<code>lastStreakDate</code>	<code>string</code>	<code>YYYY-MM-DD</code> — last day streak was updated
<code>totalSessionCount</code>	<code>number</code>	Lifetime sessions completed
<code>totalMinutesSpoken</code>	<code>number</code>	Lifetime speaking minutes
<code>subscription</code>	<code>object</code>	See Section 3.2

3.2 Subscription Sub-object (embedded in user doc)

Field	Type	Description
<code>tier</code>	<code>"free" "premium"</code>	Current access tier
<code>platform</code>	<code>"ios" "android" null</code>	Platform purchase was made on
<code>productId</code>	<code>string null</code>	StoreKit / Play Billing product ID
<code>purchaseToken</code>	<code>string null</code>	Receipt for server-side verification
<code>validUntil</code>	<code>timestamp null</code>	Expiry time for premium access
<code>autoRenewing</code>	<code>boolean</code>	Whether subscription will auto-renew

3.3 Collection: `users/{userId}/dailyUsage/{YYYY-MM-DD}`

One document per user per calendar day. Enforces the free tier 15-minute limit server-side.

Field	Type	Description
<code>date</code>	<code>string</code>	<code>YYYY-MM-DD</code>
<code>minutesUsed</code>	<code>number</code>	Total speaking minutes consumed today
<code>sessionsCount</code>	<code>number</code>	Number of sessions started today

3.4 Collection: `users/{userId}/sessions/{sessionId}`

One document per conversation session.

Field	Type	Description
<code>sessionId</code>	<code>string</code>	Auto-generated ID
<code>startedAt</code>	<code>timestamp</code>	Session start
<code>endedAt</code>	<code>timestamp null</code>	Session end (null if in progress)
<code>durationSeconds</code>	<code>number</code>	Total session length
<code>gameType</code>	<code>"none" "pronunciation_challenge" "sentence_completion"</code>	Mode for this session
<code>levelAtStart</code>	<code>number</code>	User's level at session start
<code>levelAtEnd</code>	<code>number null</code>	AI-assessed level at session end
<code>englishRatio</code>	<code>number</code>	0.0–1.0, proportion of conversation in English
<code>messageCount</code>	<code>number</code>	Total turns in session
<code>aiSummary</code>	<code>object</code>	Structured summary written by AI at session end (see 3.5)

3.5 `aiSummary` Sub-object (embedded in session doc)

This is the long-term memory input. At the start of each new session, the last 3 session summaries are passed to the AI — not raw transcripts.

Field	Type	Description
<code>assessedLevel</code>	<code>number</code>	AI's level assessment at end of session
<code>frequentErrors</code>	<code>string[]</code>	Error patterns observed (e.g. <code>"subject-verb agreement"</code>)
<code>topicsCovered</code>	<code>string[]</code>	Topics discussed (e.g. <code>"daily routine"</code> , <code>"weather"</code>)
<code>strengths</code>	<code>string[]</code>	Noted positive patterns
<code>recommendedFocus</code>	<code>string</code>	AI's suggestion for next session focus

3.6 Sub-collection: `users/{userId}/sessions/{sessionId}/messages/{messageId}`

One document per conversation turn. Used for in-session context window only — not read after the session ends.

Field	Type	Description
<code>role</code>	<code>"user" "assistant"</code>	Speaker
<code>content</code>	<code>string</code>	Transcribed text (user) or AI response text (assistant)
<code>timestamp</code>	<code>timestamp</code>	Message time
<code>languageDetected</code>	<code>"si" "en" "mixed"</code>	Language of this message
<code>pronunciationFeedback</code>	<code>string null</code>	Feedback given (assistant turns only)
<code>grammarNotes</code>	<code>string[] null</code>	Grammar issues noted

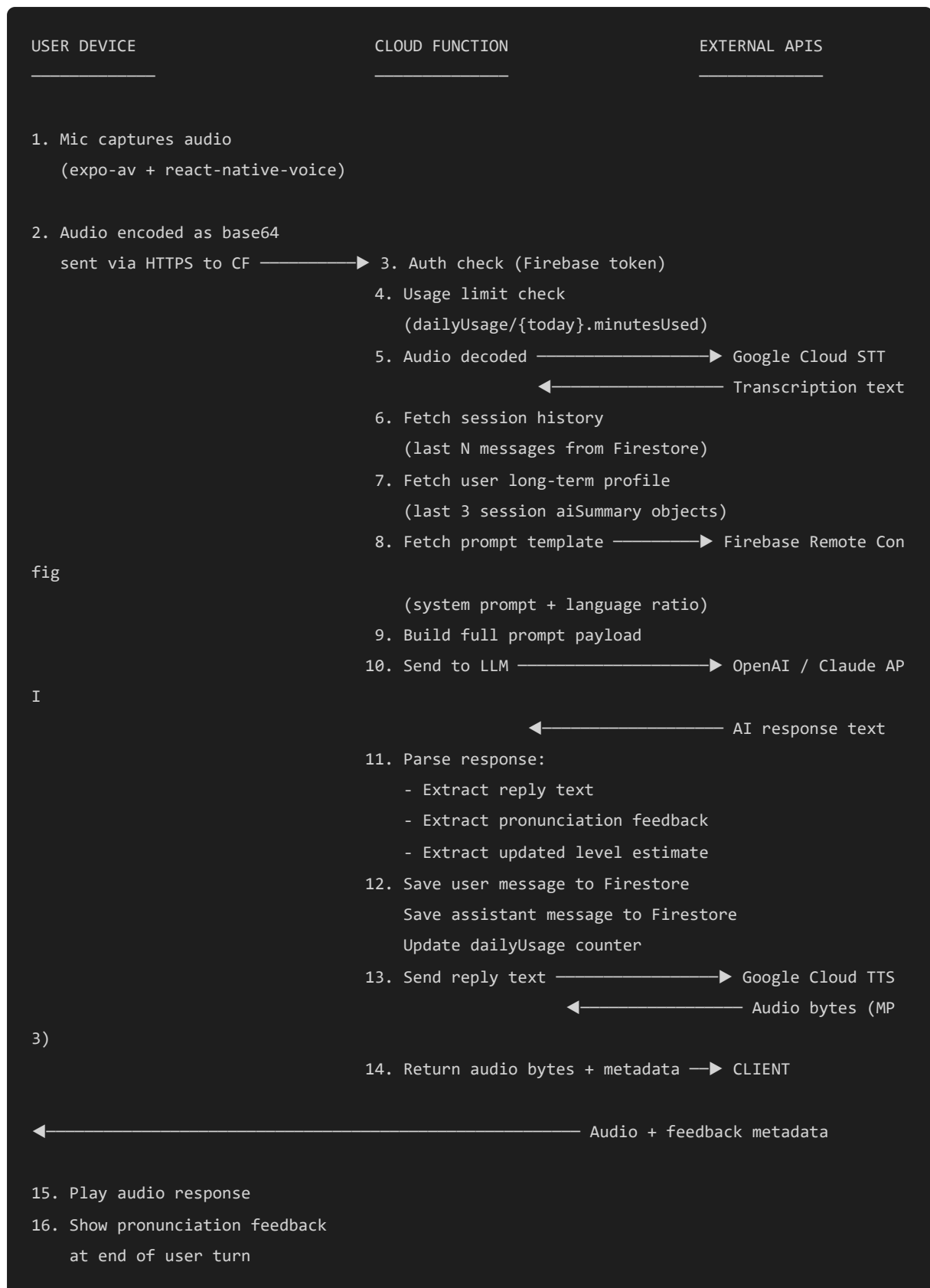
3.7 Collection: `languagePairs/{pairId}`

Config documents for supported language pairs. Enables future expansion with zero code changes.

Field	Type	Description
<code>pairId</code>	<code>string</code>	e.g. <code>"si-en"</code>
<code>nativeLanguage</code>	<code>string</code>	BCP-47 code — <code>"si"</code>
<code>targetLanguage</code>	<code>string</code>	BCP-47 code — <code>"en"</code>
<code>nativeLanguageLabel</code>	<code>string</code>	<code>"Sinhala"</code>
<code>targetLanguageLabel</code>	<code>string</code>	<code>"English"</code>
<code>sttModel</code>	<code>string</code>	Google Cloud STT model ID for this language
<code>isActive</code>	<code>boolean</code>	Whether this pair is live in production
<code>launchDate</code>	<code>timestamp</code>	When this pair was activated

4. AI Voice Pipeline

This is the critical path of the entire application. Every user interaction follows this exact sequence.



Pipeline Design Decisions

- **Audio format:** PCM 16-bit, 16kHz mono — optimal for Google Cloud STT
- **Chunk size:** Audio sent per-turn, not streamed in real-time (v1 simplicity)

- **Context window management:** Last 20 messages sent to LLM per request. Older messages are dropped but summarised in `aiSummary` at session end
- **LLM selection:** Single Cloud Function handles both OpenAI and Claude — toggled by Remote Config flag `activeLLMProvider`. Allows switching without a re-deploy
- **TTS voice:** Configurable per language pair in the `languagePairs` document — not hardcoded

5. Firebase Remote Config — Prompt Architecture

All AI behaviour is driven by Remote Config parameters. No prompt strings exist in code.

Config Key	Type	Purpose
<code>systemPrompt_si_en</code>	<code>string</code>	Full system prompt for Sinhala→English pair
<code>languageRatioRules</code>	<code>JSON</code> <code>string</code>	Rules mapping level score → English:Sinhala ratio
<code>silenceTimerSeconds</code>	<code>number</code>	How long to wait before gentle AI prompt
<code>topicRestrictions</code>	<code>JSON</code> <code>string</code>	List of approved topic areas
<code>levelAssessmentInstructions</code>	<code>string</code>	Instructions for AI to score user level
<code>pronunciationFeedbackStyle</code>	<code>string</code>	<code>"gentle"</code>
<code>activeLLMProvider</code>	<code>string</code>	<code>"openai"</code>
<code>freeTierDailyMinutes</code>	<code>number</code>	<code>15</code> — controls free limit server-side
<code>maxMessagesInContext</code>	<code>number</code>	<code>20</code> — LLM context window limit

6. Cloud Functions API Contract

All endpoints are HTTPS callable functions. Every request requires a valid Firebase Auth ID token.

6.1 `POST /session/start`

Initialises a new session document.

Request

```
{
  "gameType": "none | pronunciation_challenge | sentence_completion"
}
```

Response

```
{
  "sessionId": "string",
  "currentLevel": 0,
  "todayMinutesUsed": 0,
  "freeTierLimit": 15,
  "subscription": { "tier": "free | premium" }
}
```

6.2 `POST /session/message`

The core endpoint. Accepts audio, returns AI audio + feedback.

Request

```
{
  "sessionId": "string",
  "audioBase64": "string",
  "audioFormat": "pcm_16000hz_mono"
}
```

Server-side processing order (enforced in `processMessage.ts`):

1. Validate Firebase Auth token (`authGuard`)
2. **Validate audio payload size — reject immediately if `audioBase64` decoded size exceeds 5 MB, returning `AUDIO_FORMAT_INVALID` . This step runs before any STT call to prevent oversized chunks from reaching Google Cloud and failing silently.**
3. Check usage limit (`UsageLimiter`)
4. Decode audio → Google Cloud STT
5. Build prompt (`PromptBuilder`) → LLM → parse response
6. Google Cloud TTS → return audio

Response

```
{
  "responseAudioBase64": "string",
  "responseText": "string",
  "pronunciationFeedback": "string | null",
  "grammarNotes": ["string"],
  "updatedLevel": 0,
  "languageDetected": "si | en | mixed",
  "minutesUsedToday": 0,
  "limitReached": false
}
```

Error codes:

- `AUDIO_FORMAT_INVALID` — payload malformed or exceeds 5 MB
- `USAGE_LIMIT_REACHED` — free tier daily limit hit
- `SESSION_NOT_FOUND` — invalid sessionId
- `SUBSCRIPTION_EXPIRED` — premium expired

6.3 `POST /session/end`

Closes a session. Triggers AI summary generation and level update.

Request

```
{
  "sessionId": "string"
}
```

Response

```
{
  "durationSeconds": 0,
  "englishRatio": 0.0,
  "levelAtEnd": 0,
  "levelDelta": 0,
  "aiSummary": {
    "assessedLevel": 0,
    "frequentErrors": [],
    "topicsCovered": [],
    "strengths": [],
    "recommendedFocus": "string"
  }
}
```

6.4 `POST /subscription/verify`

Server-side receipt verification after purchase. Called immediately after StoreKit / Play Billing confirms the purchase on the client.

Request

```
{
  "platform": "ios | android",
  "productId": "string",
  "purchaseToken": "string"
}
```

Response

```
{
  "success": true,
  "tier": "premium",
  "validUntil": "ISO8601 timestamp"
}
```

6.5 GET /progress/overview

Returns the data needed to render the progress screen.

Response

```
{
  "currentLevel": 0,
  "streakDays": 0,
  "totalSessionCount": 0,
  "totalMinutesSpoken": 0,
  "recentSessions": [],
  "calendarData": {},
  "levelHistory": []
}
```

7. Mobile Application Architecture

One React Native Expo project covering both iOS and Android. TypeScript throughout. Feature-based organisation — each feature is a self-contained folder. State management with **Zustand**. Navigation with **Expo Router** (file-system based). Audio via **expo-av**. Voice input via **react-native-voice**.

App Identifiers

Identifier	Value
App name	Talk Sarasavi
iOS bundle ID	com.talksarasavi.app
Android application ID	com.talksarasavi.app
Firebase project ID	talk-sarasavi
Expo project slug	talk-sarasavi

React Native — Folder Structure

Initialised with: `npx create-expo-app@latest . --template blank-typescript`

```

apps/mobile/
|
├─ app/                                # Expo Router file-system routes
|   └─ (auth)/
|       └─ _layout.tsx
|       └─ sign-in.tsx                # Google + Apple Sign-In screen
|       └─ onboarding.tsx
|   └─ (app)/
|       └─ _layout.tsx                # Tab navigator (authenticated)
|       └─ conversation.tsx
|       └─ progress.tsx
|       └─ games.tsx
|       └─ settings.tsx
|   └─ _layout.tsx                    # Root layout – auth gate
|
├─ src/
|   └─ core/
|       └─ audio/
|           └─ audio-recorder.ts      # expo-av mic capture + silence detection
|           └─ audio-player.ts        # expo-av playback
|       └─ network/
|           └─ api-client.ts           # HTTPS wrapper + Firebase token injection
|           └─ api-error.ts           # Typed error enum mapping backend error codes
|       └─ firebase/
|           └─ auth-service.ts         # Google Sign-In + Apple Sign-In
|           └─ firestore-service.ts
|       └─ subscription/
|           └─ entitlements-service.ts # Source of truth for current user tier
|
|   └─ features/
|       └─ onboarding/
|           └─ components/
|               └─ onboarding-store.ts # Zustand store
|       └─ auth/
|           └─ components/
|               └─ GoogleSignInButton.tsx
|               └─ AppleSignInButton.tsx
|               └─ auth-store.ts
|       └─ conversation/
|           └─ components/
|               └─ ConversationScreen.tsx
|               └─ FeedbackOverlay.tsx
|               └─ conversation-store.ts
|       └─ games/
|           └─ pronunciation-challenge/
|               └─ components/
|               └─ sentence-completion/

```

```
| | | └─ components/
| | └─ progress/
| | | └─ components/
| | | └─ progress-store.ts
| | └─ settings/
| | | └─ components/
| |
| └─ shared/
|   └─ models/
|     └─ user.model.ts
|     └─ session.model.ts
|     └─ language-pair.model.ts
|   └─ constants/
|     └─ api-constants.ts
|     └─ error-codes.ts      # Mirrors shared/contracts/error.codes.ts
|   └─ hooks/
|     └─ use-auth.ts
|     └─ use-entitlements.ts
|
└─ assets/                  # App icons, splash screen, fonts
└─ app.json                 # Expo config (bundleIdentifier, package, slug)
└─ tsconfig.json
```

8. Backend (Cloud Functions) — Folder Structure

```

backend/functions/src/
|
├─ index.ts                # Exports all HTTPS callable functions
|
├─ functions/
|   └─ session/
|       ├── startSession.ts
|       ├── processMessage.ts
|       └─ endSession.ts
|   └─ subscription/
|       └─ verifyPurchase.ts
|   └─ progress/
|       └─ getOverview.ts
|
├─ services/
|   └─ ai/
|       ├── LLMService.ts      # Unified interface – OpenAI or Claude
|       ├── OpenAIProvider.ts
|       └─ ClaudeProvider.ts
|   └─ speech/
|       ├── SpeechToTextService.ts # Google Cloud STT wrapper
|       └─ TextToSpeechService.ts # Google Cloud TTS wrapper
|   └─ prompt/
|       └─ PromptBuilder.ts      # Assembles full prompt from Remote Config + context
|                                   # NOTE: system prompt token count (~1,000-2,000 tokens)
|                                   # MUST be included when calculating total context size
|                                   # against the LLM's limit – not just message history to
|
├─ kens
|   └─ usage/
|       └─ UsageLimiter.ts      # Checks and increments dailyUsage
|
├─ middleware/
|   ├── authGuard.ts           # Validates Firebase ID token on every request
|   └─ subscriptionGuard.ts    # Validates premium features are accessible
|
├─ models/
|   ├── User.ts
|   ├── Session.ts
|   ├── Message.ts
|   └─ LanguagePair.ts
|
└─ utils/

```



```
| |─ audioUtils.ts          # Base64 encode/decode helpers
| |─ dateUtils.ts          # YYYY-MM-DD helpers for dailyUsage keys
| |─ levelUtils.ts         # Level score calculation helpers
|
└─ config/
    └─ remoteConfig.ts     # Remote Config fetch + cache wrapper
```

9. Security Rules Architecture

Firestore Rules — Core Principles

```
// users/{userId} – Only the authenticated user can read/write their own document
allow read, write: if request.auth.uid == userId;

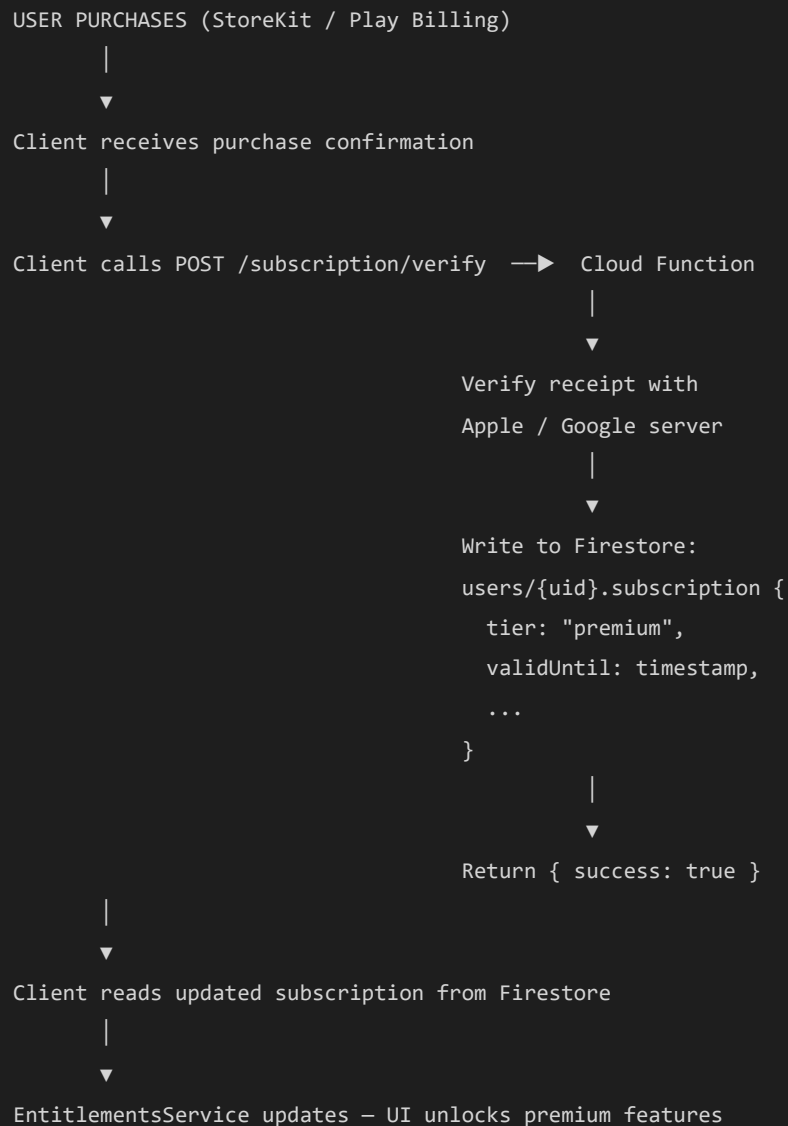
// users/{userId}/sessions/** – Same ownership rule
// users/{userId}/dailyUsage/** – Read allowed, write via Cloud Functions only

// languagePairs – Public read, no client writes
allow read: if request.auth != null;
allow write: if false;
```

[!IMPORTANT] Cloud Functions use the Firebase Admin SDK which bypasses all Firestore rules. This is intentional — the server is trusted, the client is not. Direct client writes to `dailyUsage` must be blocked to prevent free tier bypass.

10. Subscription & Entitlements System

Flow



Entitlement Check Rule

Every premium feature check goes through a single `EntitlementsService` class on the client. This class reads from a locally cached user document. The server independently validates on every AI call via `subscriptionGuard`. **Two layers, neither can be bypassed independently.**

11. Error Code Contract

Canonical error codes defined in `shared/contracts/error.codes.ts` and mirrored in mobile `src/shared/constants/error-codes.ts`.

Code	Meaning
<code>AUTH_REQUIRED</code>	No valid Firebase token
<code>SESSION_NOT_FOUND</code>	sessionId invalid or expired
<code>USAGE_LIMIT_REACHED</code>	Free tier 15-min daily limit hit
<code>SUBSCRIPTION_EXPIRED</code>	Premium subscription lapsed
<code>STT_FAILED</code>	Google Cloud STT returned an error
<code>LLM_FAILED</code>	OpenAI / Claude API call failed
<code>TTS_FAILED</code>	Google Cloud TTS returned an error
<code>INVALID_LANGUAGE_PAIR</code>	Unsupported nativeLanguage/targetLanguage
<code>AUDIO_FORMAT_INVALID</code>	Audio payload malformed
<code>RATE_LIMITED</code>	Too many requests in short window

12. Naming Conventions

Context	Rule	Example
Firestore collections	camelCase	dailyUsage , languagePairs
Firestore document fields	camelCase	validUntil , currentLevel
Cloud Function names	camelCase	startSession , processMessage
TypeScript files (backend)	PascalCase	LLMService.ts , PromptBuilder.ts
TypeScript variables (backend)	camelCase	sessionId , nativeLanguage
React Native component files	PascalCase	ConversationScreen.tsx , FeedbackOverlay.tsx
React Native non-component files	kebab-case	audio-recorder.ts , auth-store.ts
React Native components	PascalCase	ConversationScreen , GoogleSignInButton
React Native functions/variables	camelCase	startSession() , audioRecorder
Zustand store files	kebab-case	conversation-store.ts , auth-store.ts
Expo Router route files	kebab-case	sign-in.tsx , sentence-completion.tsx
Git branches	kebab-case	feature/sentence-completion-game
Environment variables	SCREAMING_SNAKE_CASE	OPENAI_API_KEY , GOOGLE_PROJECT_ID
Remote Config keys	camelCase	systemPrompt_si_en , freeTierDailyMinutes

13. Development Phases

This maps to the learning roadmap already planned. Architecture is complete before Phase 5.

Phase	Focus	Deliverable
Phase A	Repo setup, Firebase project, CI skeleton	Monorepo initialised; Expo app created via <code>cd apps/mobile && npx create-expo-app@latest . --template blank-typescript</code> (command must be run from inside <code>apps/mobile/</code> — not from the monorepo root — to ensure Expo files are created in the correct location); Firebase dev/prod projects created (<code>talk-sarasavi-dev</code> , <code>talk-sarasavi</code>); bundle ID <code>com.talksarasavi.app</code> set in <code>app.json</code> ; GitHub Actions linting
Phase B	Auth + User profile	Google Sign-In + Apple Sign-In flows; Firestore user doc creation on first sign-in; <code>languagePairs</code> collection seeded with <code>si-en</code> config
Phase C	Voice pipeline (backend)	STT → LLM → TTS Cloud Function working end-to-end in emulator; 5 MB payload guard and PromptBuilder token accounting in place
Phase D	Conversation UI (mobile)	Mic capture via expo-av, audio upload, playback, feedback display — no games yet
Phase E	Progress tracking	Session end flow, aiSummary generation, progress screen
Phase F	Freemium enforcement	dailyUsage counter, USAGE_LIMIT_REACHED gate, paywall UI
Phase G	Subscription system	StoreKit + Google Play Billing, receipt verification, entitlements unlock
Phase H	Games (premium)	Pronunciation Challenge + Sentence Completion modes
Phase I	Polish + launch prep	Onboarding, streak system, app store assets, production Firebase rules

Resolved Decisions

[!NOTE] **Q1 — Cloud Functions runtime:**  **Resolved.** TypeScript (Node.js) confirmed.

[!NOTE] **Q2 — Authentication method:**  **Resolved.** Google Sign-In + Apple Sign-In at launch. No email/password in v1. Apple Sign-In is required by App Store policy since Google Sign-In is offered. Both providers implemented in `auth-service.ts` and surfaced via `GoogleSignInButton` and `AppleSignInButton` components.

[!NOTE] **Q3 — App name:**  **Resolved.** App name is **Talk Sarasavi**. iOS bundle ID: `com.talksarasavi.app`. Android application ID: `com.talksarasavi.app`. Firebase project ID: `talk-sarasavi`. Expo slug: `talk-sarasavi`.

Open Questions

[!NOTE] **Q4 — Speaking games spec:** The architecture has placeholder folders for `pronunciation-challenge/` and `sentence-completion/`. Before Phase H, a detailed spec is required covering: how a round starts, what the AI generates as the challenge, how user response is evaluated, and how scoring works. This does not block Phases A–G.

End of architecture blueprint — ready for review before any code is written.